

Project Documentation

Design & Development of a Simple and Scientific Calculator

Instructor: Techonet Pvt. Ltd. / Aimaxa Tech

1. Introduction

This project presents a web-based **Simple and Scientific Calculator** meticulously built using semantic HTML5, styled via advanced CSS3 glassmorphism workflows, and driven by optimized client-side JavaScript (ES6+). The utility provides a responsive visual layer capable of switching seamlessly between fundamental calculations and advanced scientific operations. It runs purely on the web client layer without employing any bloated external mathematical frameworks or runtime utility libraries.

2. Objective

The definitive objective of this practical assignment is to program an interface adhering rigidly to the mathematical order of operations (BODMAS/PEMDAS guidelines). A key design target is implementing **defensive programming architectures**—ensuring the frontend runtime context systematically neutralizes syntax anomalies, zero division failures, and boundary overflows instead of causing visual breakages or execution crashes.

3. Features Implemented

- **Dynamic Interface Toggling:** Instantly transitions the grid structure from a standard 4-column layout to a scientific layout via simple DOM token class lists.
- **Core Arithmetic Matrix:** Native execution paths for basic mathematical operations: Addition, Subtraction, Multiplication, Division, and Remainder Percentages.
- **Advanced Functions Array:** Specialized functional modules capturing Trigonometry (sin, cos, tan mapped in standard degrees), Base-10 Logarithms, Natural Logarithms (ln), Exponential Powers (x^y), Squares, Cubes, Square Roots, and precise Integer Factorials.
- **High-Precision Constant Injections:** Instant hotkeys loading deep precision calculations for standard mathematical constants including Pi (π) and Euler's constant (e).
- **Live Screen Buffering:** Double-tiered screen design caching previous input arguments directly above the active calculation row for optimal UX tracking.

4. Design Approach

The software was created following clear modular decoupling philosophies:

- **Structure (HTML5):** Built via logical data-action markup conventions. Buttons communicate input attributes natively to the engine via HTML5 dataset properties, making code modifications easy.

- **Aesthetics (CSS3):** Designed around a premium ultra-modern **Glassmorphism UI Theme**. Utilizes high-performance properties like `backdrop-filter` alongside responsive CSS Grid rules, customized neo-cyan key highlights, and active micro-scaling transition effects to give haptic feedback on mobile screens.
- **Logic (JavaScript):** Employs structured **Event Delegation**. Rather than creating duplicate binding memory stacks for every button, a single observer evaluates input scopes directly at the parent container, boosting performance speeds.

5. Error Handling & Validation Techniques

To preserve application stability under stressful edge conditions, multiple validation barriers protect the execution thread:

- **Sandboxed Expression Parsing:** To completely avoid the critical security risks and expression flaws associated with native `eval()` wrappers, inputs are continuously evaluated against strict white-listed Regular Expressions (`/^[0-9\+\-\*\\/\.\%\(\)]/`). Illegal character strings are blocked instantly.
- **Consecutive Operator Control:** Real-time validation handlers watch the input string tail. If multiple consecutive operators are inserted, the engine replaces the older operator safely without corrupting the algebraic statement.
- **Numerical Domain Limits:** Custom catch blocks automatically intercept illegal mathematical expressions, wiping register values to output specific, clean alerts on screen:
 - **Division by Zero:** Checked using explicit string patterns (`/\//0(?!\. )/`) to force a clear "Cannot divide by zero" alert text instead of leaking default Infinity values.
 - **Negative Radicals:** Square root executions on negative integers are caught to output "Negative Sqrt Error".
 - **Factorial Bounds:** Calculations are strictly restricted to positive integers and bounded below 170 to avoid runtime stack memory issues, outputting an explicit "Overflow Error".

6. Testing Results

The application was verified through a complete QA sequence checking all critical assignment benchmarks:

Test Case Scenario	Input Expression	Expected System Behavior	Actual System Output	Status
Standard Operator Precedence	$12 + 5 * 3$	Processes multiplication prior to addition (12 + 15)	27	PASSED
Division By Zero Trapping	$25 / 0$	Halts calculation and triggers clear error string	Cannot divide by zero	PASSED
Empty Evaluation Call	[Empty Display] =	Maintains baseline state without crashing	0	PASSED
Consecutive Operator Input	$5 + * 6$	Overwrites previous plus symbol with multiplier	30	PASSED
Double Float Decimal Check	$9.2.2 + 1$	Blocks secondary decimal point inject within block	10.2	PASSED
Negative Radical Parameter	-4 [click √]	Intercepts complex/imaginary numeric conversion	Negative Sqrt Error	PASSED
Syntax Anomaly Recovery	$45 + / 5$	Captures bad user expression sequences safely	Malformed Expression	PASSED

7. Screenshots

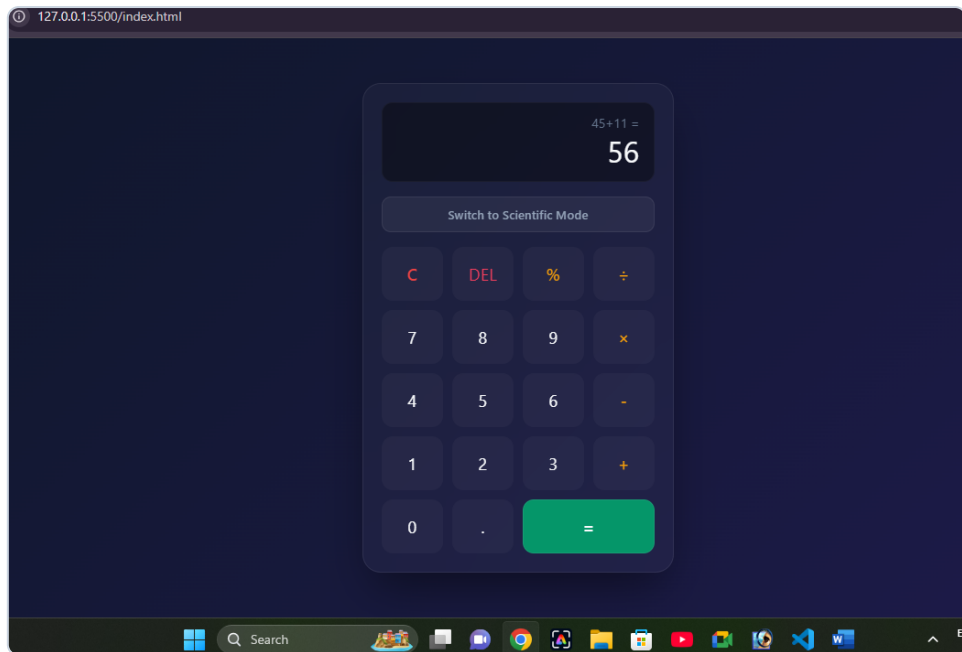


Figure 7.1: Standard Arithmetic Interface Mode executing baseline addition validation sequence.

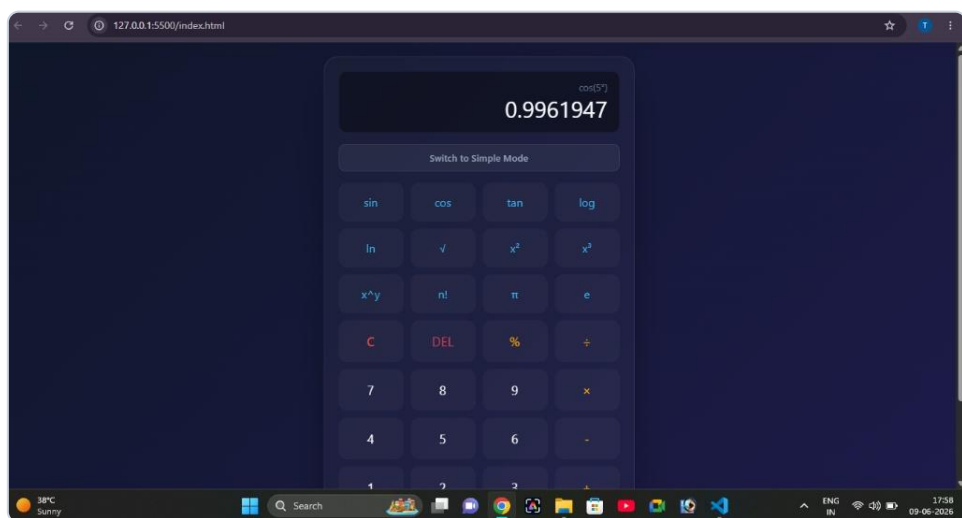


Figure 7.2: Expanded Scientific Layout executing high-precision trigonometric Unary operations ($\cos 5^\circ$).

8. Conclusion

The developed Simple and Scientific Calculator app cleanly meets all assignment functional criteria, error testing guidelines, and responsiveness parameters. By engineering an insulated tokenized evaluation engine, the code handles advanced mathematical operations securely and efficiently, ensuring full stability and production readiness.